

\Chương 4. ĐỒ THỊ

4.1.Khái niệm

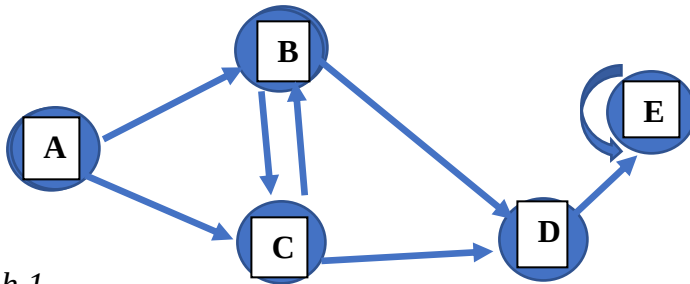
Cấu trúc dữ liệu đồ thị (graph) là kiểu cấu trúc tổng quát hơn cấu trúc cây đã học, sơ đồ mạng lưới (network) là một dạng ứng dụng biến thể của đồ thị.

Một đồ thị ở dạng biểu diễn hình ảnh là các cặp nút và cạnh nối với nhau.

Một đồ thị ở dạng biểu diễn toán học là một tập hợp (V, E) , trong đó V (vertices) là tập các nút và E (edges) là tập các cạnh kết nối các cặp nút.

Một đồ thị mà các cạnh nối nút không có hướng gọi là đồ thị vô hướng, các cạnh có hướng gọi là đồ thị có hướng.

Ví dụ. trong đồ thị dưới đây, nút là tập $V = \{A, B, C, D, E\}$, cạnh là tập $E = \{AB, AC, BC, BD, CD, DE\}$



Hình 1.

Các khái niệm :

- Nút kề nhau : 2 nút được nối với nhau bằng 1 cạnh. Ví dụ A kề B, A kề C,..., D kề E.
- Đường : biểu diễn một dãy các cạnh nối liên các nút. Trong hình trên, ACDE hoặc ABDE biểu diễn một đường từ A tới D.

4.2.Đồ thị có hướng

4.2.1.Khái niệm

Khi các nút nối với nhau bằng các cạnh có hướng thì ta có một đồ thị có hướng. Kiểu cấu trúc cây – Tree – đã xét trước đây là một dạng riêng của đồ thị có hướng, trong cấu trúc cây, các nút luôn xét theo hướng đi từ nút gốc đến các nút lá. Nút gốc của cây không có cạnh nào đi đến, mỗi nút lá chỉ có duy nhất một đường đi từ gốc đến nó. Trong đồ thị trên, A là nút gốc, E là nút lá và có 3 đường có thể đi từ A $\rightarrow$ E là ABDE, ACDE, ABCDE. Khác với Cây, trong đồ thị có hướng, có thể ở một nút có hướng quay vòng tại nút đó (giống hình ảnh một bùng binh, vòng xoay ở nút giao thông có nhiều đường giao nhau).

Đồ thị có hướng được áp dụng nhiều trong thực tế, thành một lĩnh vực riêng là sơ đồ mạng lưới, được vận dụng giải các bài toán như quản lý thi công công trình xây dựng, quản lý mạng lưới điện, lập kế hoạch sản xuất kinh doanh của một doanh nghiệp...

4.2.2. Cài đặt đồ thị có hướng

a) Ma trận kề

Cách cài đặt thông dụng là dùng ma trận kề của một đồ thị có n nút, là một ma trận vuông A có n hàng n cột $A[n][n]$, trong đó :

$$A[i][j] = \begin{cases} 1 & \text{nếu có cạnh nối nút } i \text{ đến nút } j \\ 0 & \text{nếu không có cạnh nối nút } i \text{ đến nút } j \end{cases}$$

A – Adjacency matrix – ma trận kề

Như vậy, với đồ thị hình 1 ta lập ma trận kề :

$$A[] = \begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 1 & 0 & 0 \\ B & 1 & 0 & 1 & 1 & 0 \\ C & 0 & 1 & 0 & 1 & 0 \\ D & 0 & 0 & 0 & 0 & 1 \\ E & 0 & 0 & 0 & 0 & 1 \end{array}$$

b) Mảng giá trị các nút

Dùng mảng $D[]$ lưu dữ liệu các nút, $D[i]$ là dữ liệu của nút i .

c) Danh sách kề

Trong ma trận kề thường có nhiều phần tử $= 0$, nếu lưu trữ tất cả sẽ lãng phí ô nhớ. Có cách khác lưu trữ thông tin của ma trận kề tiết kiệm bộ nhớ bằng cách dùng danh sách liên kết theo hàng, mỗi nút sẽ là một mảng liên kết, được duy trì bởi một con trỏ trỏ đến nút đầu của nút đó, mỗi phần tử tiếp theo là một nút có 2 thành phần : dữ liệu và liên kết, được đi ra từ cùng một nút.

Ký hiệu $V[x]$ là con trỏ cho nút x .

$V[A] \rightarrow \begin{array}{|c|c|} \hline A & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline B & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline C & \text{NULL} \\ \hline \end{array}$

$V[B] \rightarrow \begin{array}{|c|c|} \hline B & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline C & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline D & \text{NULL} \\ \hline \end{array}$

$V[C] \rightarrow \begin{array}{|c|c|} \hline C & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline D & \text{NULL} \\ \hline \end{array}$

$V[D] \rightarrow \begin{array}{|c|c|} \hline D & \text{next} \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline E & \text{NULL} \\ \hline \end{array}$

$V[E] \rightarrow \begin{array}{|c|c|} \hline E & \text{NULL} \\ \hline \end{array}$

Nếu thay chữ cái tên nút bằng số thứ tự nút 1,2,... thì ta danh sách kề có dạng :

V[1] ->

1	next
---	------

 ->

2	next
---	------

 ->

3	NULL
---	------

V[2] ->

2	next
---	------

 ->

3	next
---	------

 ->

4	NULL
---	------

V[3] ->

3	next
---	------

 ->

4	NULL
---	------

V[4] ->

4	next
---	------

 ->

5	NULL
---	------

V[5] ->

5	NULL
---	------

Hình 2. Danh sách kề

Để thiết lập danh sách liên kề như hình 2 trên, có các bước :

1) Khai báo mảng các con trỏ V[], mỗi phần tử có 2 thành phần du_lieu và next

- n là số nút tối đa của đồ thị

2) Khởi tạo nút đầu tiên của nút i có dữ liệu, next được gán NULL

{ V[i].du_lieu = du_lieu0; V[i].next = NULL }

Với mỗi nút i sẽ thêm các nút kế tiếp theo sơ đồ như hình 2 cho đến hết, theo hàm thêm nút vào cuối danh sách liên kết đã học.

3) Tăng i và lặp lại bước 2 cho đến khi i=n.

4.2.3. Duyệt tìm theo chiều sâu trên đồ thị có hướng

Ý tưởng của duyệt tìm theo chiều sâu

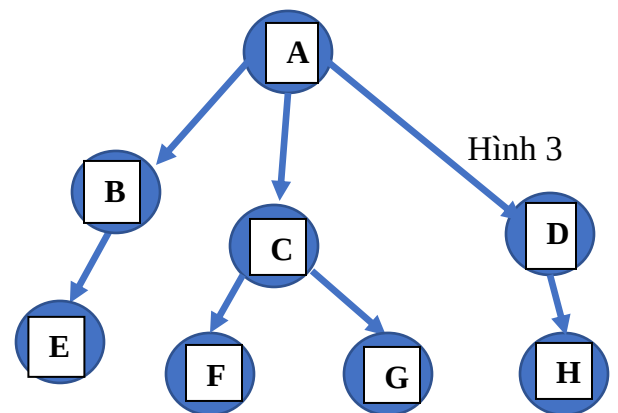
là từ gốc đi xuống nút con, thăm nút con của nút con...cho đến khi hết các cấp dưới của nút con mới quay lên duyệt nút con khác cho đến khi tìm được.

Với đồ thị ở hình 3, đầu tiên thăm nút A,

sau đó thăm nút con B, thăm nút con E,

quay lên thăm nút con C, thăm tiếp các nút con của C là F,G; quay lên thăm nút D, thăm tiếp nút con của B là H.

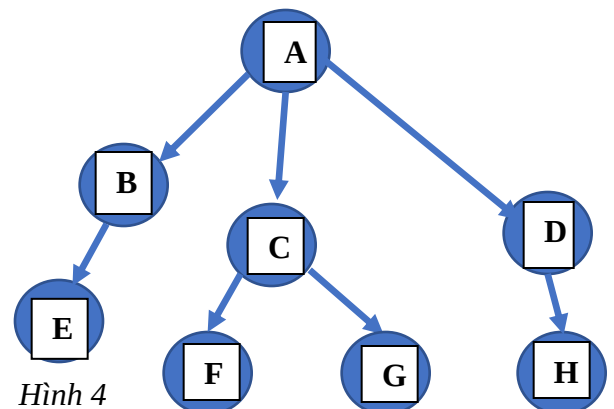
Thông thường dùng cấu trúc ngăn xếp – stack để tổ chức duyệt đồ thị theo chiều sâu.



Hình 3

5.2.4. Duyệt tìm theo chiều ngang trên đồ thị có hướng

Theo cách duyệt này ta đi theo chiều ngang của từng cấp.



Hình 4

Ví dụ : với đồ thị ở hình 4, đầu tiên đi từ gốc A, sau đó thăm các nút mức 1 B, C, D sau đó thăm các nút mức 2 E, F, G, H.

Cách này giống như duyệt hết các nút ở mức thứ i rồi mới duyệt ở mức thứ i+1 trong cây nhị phân. Thông thường dùng cấu trúc hàng đợi – queue để tổ chức duyệt đồ thị theo chiều ngang

4.3. Đồ thị vô hướng

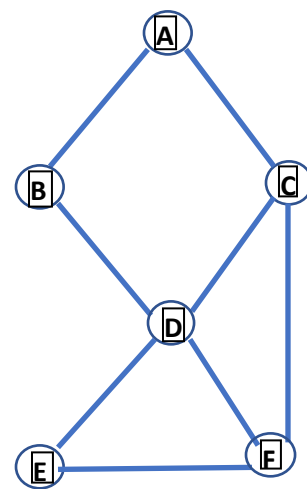
Khi các cạnh trong đồ thị không chỉ hướng thì ta có đồ thị vô hướng. Trong đồ thị vô hướng không có một nút nào được nối với chính nó (*không có xoay vòng*).

Cài đặt đồ thị vô hướng cũng gần giống như đối với đồ thị có hướng, dùng ma trận kề, danh sách các cạnh.

4.3.1. Cài đặt đồ thị vô hướng bằng ma trận kề

Cho đồ thị như hình 5.

Ma trận kề bây giờ có dạng :

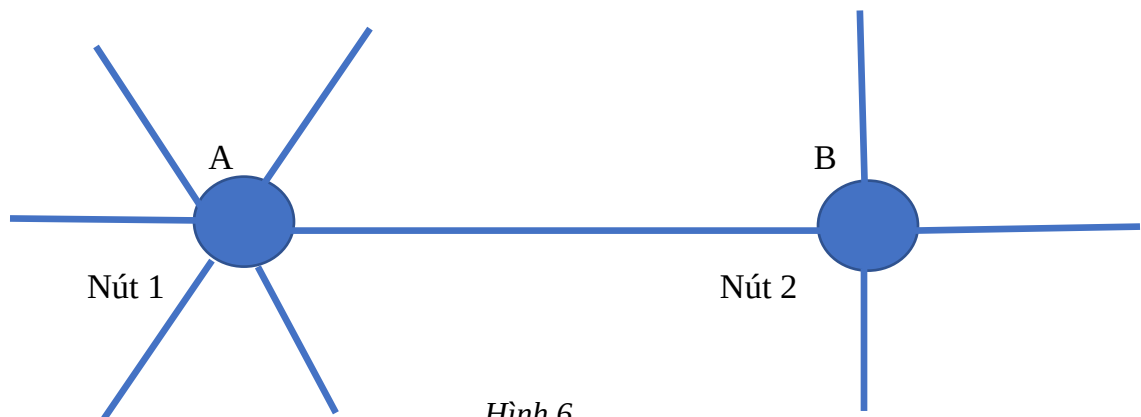
$$A[] = \begin{matrix} & \begin{matrix} A & B & C & D & E & F \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} & \begin{vmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{vmatrix} \end{vmatrix}$$


Hình 5

Nhận xét : ma trận kề của đồ thị vô hướng luôn đối xứng qua đường chéo chính.

4.3.2. Cài đặt đồ thị vô hướng bằng danh sách các cạnh

Trong đồ thị vô hướng, mỗi nút ở đầu cạnh có thể nối với nhiều nút khác. Chẳng hạn khi biểu diễn mạng lưới các đường phố của một khu phố, có những đoạn đường mà một đầu là ngã 6, một đầu là ngã tư :

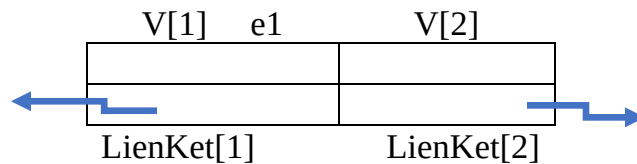


Hình 6.

Danh sách các cạnh là khi ta đánh số thứ tự các nút, mỗi cạnh sẽ được lưu trữ thông tin dạng CẠNH – NÚT (Edge Node), sẽ lưu nút V[i] mà cạnh được nối tới. Trong hình 6 trên, cạnh nối A-B là cạnh

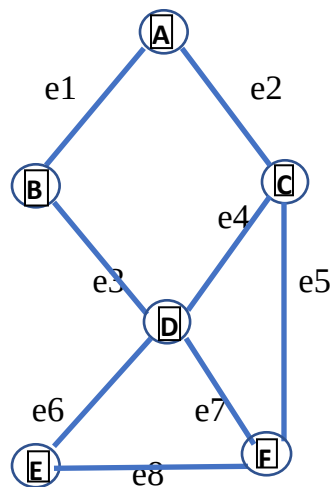
nút nút $V[1]$ và $V[2]$; đồng thời phải hiểu $V[1]$ là điểm cuối của cạnh khác và $V[2]$ cũng vậy. Ta dùng mảng các con trỏ $V[i]$, mỗi con trỏ $V[i]$ trỏ đến nút đầu lưu trữ dữ liệu của các nút, nút đầu $V[i]$ này có trường liên kết trỏ đến một cạnh- nút mà có nút i là một trong các điểm cuối của nó.

Mỗi phần tử e_i trong danh sách các cạnh có dạng :



Với đồ thị ở hình 5 trên, ta có danh sách cạnh như sau:

- 1.AB
- 2.AC
- 3.BD
- 4.CD
- 5.CE
- 6.DE
- 7.DF
- 8.EF



Hình 7

Chú ý : một cạnh đã được kê khai thì không kê khai lại, ví dụ đã khai báo cạnh AB thì không khai BA nữa.

4.4.Các thuật toán cơ bản trên đồ thị

Trong lý thuyết đồ thị, có 2 kiểu duyệt cơ bản là duyệt theo chiều ngang và duyệt theo chiều sâu. Duyệt đồ thị theo chiều ngang (*breadth-first searching* BFS) gần giống với duyệt cây nhị phân theo các mức trong cây, duyệt đồ thị theo chiều sâu(*depth-first searching* DFS) gần giống với duyệt cây theo tiền tự DLR; Chỉ có chỗ khác khác biệt là trong đồ thị không có nút gốc, việc duyệt có thể bắt đầu từ một nút bất kỳ.

4.4.1.Thuật toán duyệt đồ thị theo bề ngang – BFS

Ý tưởng của thuật toán duyệt đồ thị theo bề ngang là duyệt các nút(nút) của đồ thị theo các mức theo khoảng cách xa dần nút gốc(nút được khởi đầu duyệt), mức sau cách mức trước 1 cạnh tính từ nút bắt đầu.

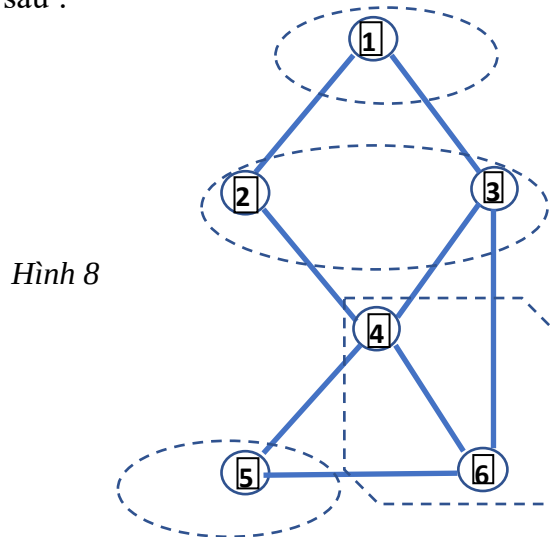
Giải thích thuật ngữ : trong các thuật toán BFS và DFS có sử dụng cụm từ “đã duyệt”, một số tài liệu dùng với ý nghĩa không rõ ràng, dễ nhầm lẫn. Trong bài này nói “đã duyệt” là đã có một tác động vào nút đó, chẳng hạn đã đánh dấu “đã duyệt”, đang thực hiện lập tập nút liên kề của nó, xét xem các nút trong tập liên kề của nó đã được duyệt chưa để đưa vào hàng đợi...

4.4.1.1.Giới thiệu thuật toán

Xét bài toán : cho một đồ thị vô hướng không có trọng số $G=(V,E)$, trong đó V là tập các nút(nút), E là tập các cạnh; có một nút khởi đầu $s \in V$.

Để duyệt đồ thị theo BFS, sẽ sử dụng cấu trúc hàng đợi $Q[]$ và danh sách chưa được sắp là dãy $bfs[]$ cài đặt kiểu mảng array. Hàng đợi $Q[]$ dùng cho việc thăm qua các nút, lưu các nút đã được thăm và chờ đợi duyệt, nút nào được đánh dấu “đã duyệt” sẽ xóa khỏi hàng đợi rồi sau đó mới thực hiện các thao tác liên quan đến nó. Mảng $bfs[]$ để lưu kết quả, chỉ số của mảng $bfs[i]$ là số nút, còn giá trị của $bfs[i]$ sẽ là thứ tự duyệt của nút đó, $bfs[i]=0$ là nút i chưa được duyệt, $bfs[i] \neq 0$ là đã duyệt, ví dụ $bfs[6]=5$ có nghĩa nút số 6 có thứ tự duyệt là 5.

Cho một đồ thị như sau :



Thuật toán duyệt đồ thị theo bề ngang là duyệt các nút(nút) của đồ thị theo các mức theo khoảng cách xa dần nút gốc(nút được khởi đầu duyệt), mức thứ $i+1$ cách xa nút gốc hơn mức thứ i một cạnh. Với đồ thị ở hình 1, mức 1 chỉ có nút khởi đầu 1; mức 2 có nút 2 và 3, mức 3 có nút 4 và 6, mức 4 có một nút 5.

Thuật toán được cài đặt theo cấu trúc hàng đợi, nút nào được duyệt trước sẽ ra khỏi hàng đợi trước – cấu trúc FIFO (First In, First Out),

4.4.1.2. Các bước của thuật toán BFS

Đầu vào : đồ thị $G=(V,E)$, V là tập các nút, E là tập các cạnh.

Đầu ra : đồ thị $G_{\text{kết quả}}=(V,E)$, với điều kiện mỗi nút của đồ thị kết quả được nối với nút **s** khi và chỉ khi nó đã được đánh dấu “đã duyệt”.

1. Thiết lập hàng đợi $Q[]$ và danh sách $bfs[]$, khởi tạo nút s vào Q ; mảng $bfs[]$ sẽ khởi tạo $bfs[1]=\dots=bfs[n]=0$, n là số nút; đặt biến k chỉ thứ tự duyệt của nút, k bắt đầu từ 0; sau này, nút i nào đã duyệt thứ k thì $bfs[i]$ được gán bằng k : $bfs[i]=k$.

Lúc đầu, Đánh dấu s là nút “đã duyệt”, tất cả các đỉnh còn lại là chưa duyệt.

2. while (Q không rỗng)

{Đối với nút hiện thời đang duyệt(Chính là nút đầu hàng đợi Q) :

Duyệt tiếp tới các nút liền kề, đánh dấu và đưa chúng vào $Q[]$ và $bfs[]$;

Nếu đã hết nút liền kề, thì xóa nút đầu hàng đợi $Q[]$ (nút hiện thời đang duyệt).

Lặp lại với nút mới đầu hàng đợi

}

Với đồ thị ở hình 8, từng bước thực hiện như sau.

Tạo dãy [6] và hàng đợi Q[6]

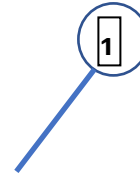
Bước 0. Đưa nút khởi đầu 1 vào Q

BFS

--	--	--	--	--	--

Q

1					
---	--	--	--	--	--



Bắt đầu từ đây sẽ thực hiện các bước lặp trong vòng lặp **while** sau đây cho đến khi hàng đợi Q[] rỗng :

k=0; k lưu thứ tự duyệt của nút.

while (!Empty(Q[])){Trong vòng lặp này thực hiện các lệnh :

- Lấy ra nút đầu Q đặt là u (chưa xóa u ở hàng đợi) ; khởi đầu là u=1.
- Xóa u khỏi hàng đợi Q[];
- Kiểm tra xem u đã duyệt chưa, nếu đã duyệt (bfs[u] ≠ 0) thì bỏ qua các phần sau, lặp lại **while**
- Duyệt u, đánh dấu u đang duyệt, thứ tự duyệt k=k+1 ⇒ bfs[u]=k; (dùng nét màu gạch chéo đánh dấu hoặc ra màn hình);
- Tạo tập các nút liền kề với u dưới dạng danh sách lienke(u);
- Lần lượt xét từng nút v thuộc lienke(u), nếu v chưa được duyệt (bfs[v]=0) sẽ đưa v vào Q[];

}

Lần lặp 1. Lấy nút 1 đầu hàng đợi, duyệt nút 1,

Xóa nút 1 khỏi hàng đợi Q[];

Kiểm tra xem u đã duyệt chưa, nếu đã duyệt (bfs[u] ≠ 0) thì bỏ qua các phần sau, lặp lại **while**.

k=k+1=1. Ghi thứ tự duyệt nút 1 vào bfs[1]=1,

bfs

1	2	3	4	5	6
1	0	0	0	0	0

Tạo tập các nút liền kề lienke(1)={2,3} (nút 1 nối với đỉnh 2 , với nút 3),
Lần lượt xét 2 nút này, chúng chưa được duyệt(bfs[2]=0, bfs[3]=0) nên đưa 2 đỉnh này vào hàng đợi Q[], vẽ thêm nút 2,3 vào cây :

Q

	2	3			
--	---	---	--	--	--

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**.

Lần lặp 2. Lấy nút 2 ở đầu hàng đợi Q[]. Duyệt nút 2, Xóa nút 2 khỏi hàng đợi Q[];

Kiểm tra xem nút 2 đã duyệt chưa, nếu đã duyệt (bfs[u] ≠ 0) thì bỏ qua các phần sau, lặp lại **while**.

Nút 2 chưa duyệt ⇒ Ghi thứ tự duyệt nút 2 vào bfs[2]=k+1=2;

bfs

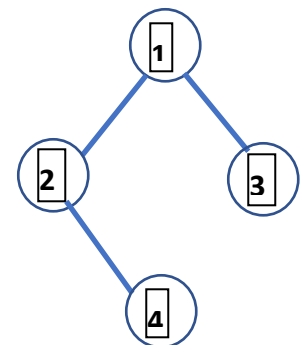
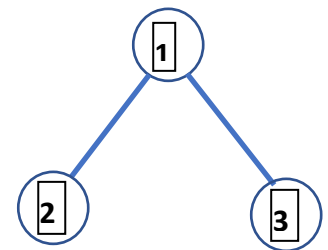
1	2	3	4	5	6
1	2	0	0	0	0

Tạo tập các nút liền kề lienke(2)={4} (nút 2 có cạnh nối 2-4, vẽ cạnh 2-4).
Xét nút 4 trong danh sách kề lienke(2), nó chưa được duyệt (bfs[4]=0) nên đưa 4 vào hàng đợi Q[]:

Q

3	4				
---	---	--	--	--	--

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**



Lần lặp 3. Lấy nút 3 từ Q[]. Duyệt nút 3, xóa nút 3 khỏi Q[].

Kiểm tra xem u đã duyệt chưa, nếu đã duyệt ($bfs[u] \neq 0$) thì bỏ qua các phần sau, lặp lại **while**

Nút 3 chưa duyệt => Ghi thứ tự duyệt nút 3 vào $bfs[3]=k+1=3$;

bfs	1	2	3	4	5	6
	1	2	3	0	0	0

Tạo tập các nút liền kề $lienke(3)=\{4,6\}$. Xét nút 4,6 trong danh sách kề $lienke(3)$, chúng chưa được duyệt ($bfs[4]=0$, $bfs[6]=0$) nên đưa 4,6 vào Q[]:

Q	4	6		
---	---	---	--	--

Trên đồ thị dùng nét đứt nối 3-4 còn nút 6 bây giờ mới xét đến nên cạnh 3-6 vẽ nét liền.

Đã xét tất cả các nút $v \in lienke(3)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 4. Lấy nút 4 từ Q[]. Duyệt nút 4, xóa nút 4 khỏi Q[].

Kiểm tra xem 4 đã duyệt chưa, nếu đã duyệt ($bfs[u] \neq 0$) thì bỏ qua phần sau, lặp lại **while**.

Nút 4 chưa duyệt => Ghi thứ tự duyệt nút 4 vào $bfs[4]=k+1=4$;

đánh dấu / nút 4 đang duyệt trong hàng đợi Q[],

bfs	1	2	3	4	5	6
	1	2	3	4	0	0

Tạo tập các nút liền kề $lienke(4)=\{5,6\}$.

Xét nút 5,6 trong danh sách kề $lienke(4)$, chúng chưa được duyệt ($bfs[5]=0$, $bfs[6]=0$) nên đưa 5, 6 vào cuối hàng đợi Q[].

Cạnh 4-5 vẽ nét liền, còn 4-6 dùng nét đứt nối vì cạnh 3-6 đã nối liền từ nút 3.

Q	6	5	6
---	---	---	---

Đã xét tất cả các nút $v \in lienke(4)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 5. Lấy nút 6 từ Q[]. Duyệt nút 6, xóa nút 6 khỏi Q[] :

Q	5	6
---	---	---

Kiểm tra xem nút 6 đã duyệt chưa, nếu đã duyệt ($bfs[u] \neq 0$) thì bỏ qua các phần sau, lặp lại **while**.

bfs	1	2	3	4	5	6
	1	2	3	4	0	0

Nút 6 chưa duyệt => Ghi thứ tự duyệt nút 6 vào $bfs[6]=k+1=5$;

bfs	1	2	3	4	5	6
	1	2	3	4	0	5

Tạo tập các nút liền kề $lienke(6)=\{5,4,3\}$.

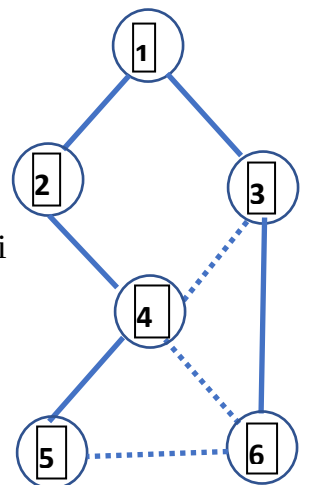
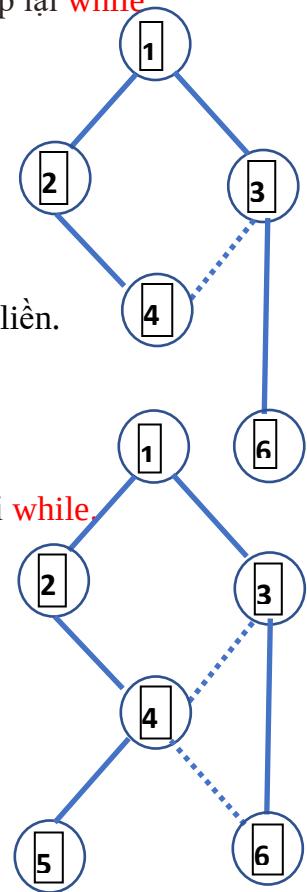
Xét các nút trong danh sách kề $lienke(6)$, chỉ có nút 5 chưa duyệt $bfs[5]=0$ nên đưa nó vào Q[]. Nút 6 có cạnh nối 6-5, 6-4, 6-3, nút 3, 4 đã duyệt, nút 5 đã được nối từ nút 4, nên cạnh 6-5 ta dùng nét đứt.

Q	5	6	5	
---	---	---	---	--

Đã xét tất cả các nút $v \in lienke(6)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 6 : Lấy nút 5 từ Q[]. Duyệt nút 5, xóa nút 5 khỏi Q[].



Q

6	5		
---	---	--	--

Kiểm tra xem 5 đã duyệt chưa, nếu đã duyệt ($bfs[5] \neq 0$) thì bỏ qua các phần sau, lặp lại **while**

Nút 5 chưa duyệt => Ghi thứ tự duyệt nút 5 vào $bfs[5]=k+1=6$;

bfs

1	2	3	4	5	6
1	2	3	4	6	5

Tạo tập các nút liên kề $lienke(5)=\{4,6\}$.

Xét tất cả các nút $v \in lienke(5)$ đã duyệt hết nên không thêm vào Q[]

Kiểm tra, Q chưa Rỗng nên kết thúc vòng **while**

Lần lặp 7 : Lấy nút 6 từ Q[], xóa nút 6 khỏi Q[].

Q

5

Kiểm tra xem 6 đã duyệt chưa, nếu đã duyệt ($bfs[6] \neq 0$) thì bỏ qua các phần sau, lặp lại **while**

Kiểm tra, Q chưa Rỗng nên lặp vòng **while**

Lần lặp 8 : Lấy nút 5 từ Q[], xóa nút 5 khỏi Q[].

Q

--

Kiểm tra xem 5 đã duyệt chưa, nếu đã duyệt ($bfs[5] \neq 0$) thì bỏ qua các phần sau, lặp lại **while**

Kiểm tra, Q Rỗng nên kết thúc vòng **while**

Danh sách kết quả $bfs[]$ chứa các nút theo thứ tự bề rộng từ điểm bắt đầu sẽ cho một cây nhị phân đặc biệt là **Cây khung - Spanning Tree**, có tính chất : là một tập con của Graph G mà có tất cả các nút được bao bởi số cạnh tối thiểu, một cây khung sẽ không hình thành một vòng tuần hoàn và nó cũng không thể bị ngắt giữa chừng

Khi ta bỏ các cạnh có nét đứt, đồ thị kết quả là cây khung (spanning tree):

Đồ thị kết quả có tính chất :

Mỗi nút(nút) của đồ thị kết quả

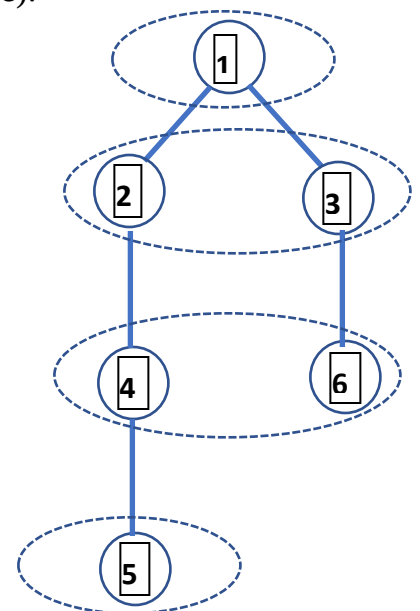
có thể đi đến từ nút khởi đầu khi

và chỉ khi nó được đánh dấu “đã duyệt”

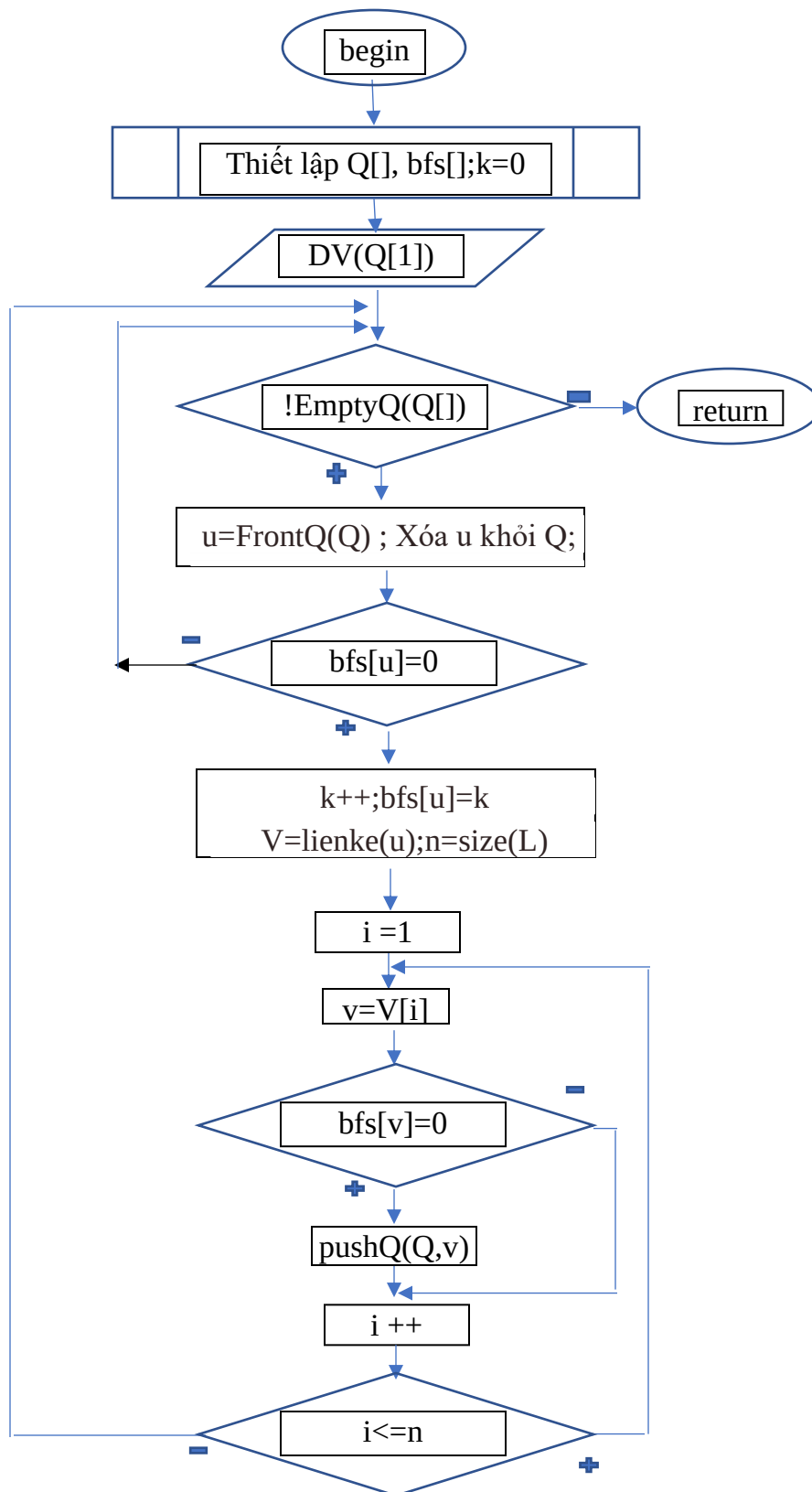
Duyệt đồ thị theo BFS sẽ cho ta hình ảnh

duyet cây theo từng mức, từ mức 1 đến mức

cuối cùng, mức sau cách mức trước 1 cạnh.



4.4.1.3. Sơ đồ thuật toán BFS :



Danh sách kết quả chứa các nút theo thứ tự bề rộng từ điểm bắt đầu sẽ cho một cây nhị phân đặc biệt là **Cây khung - Spanning Tree**, có tính chất : là một tập con của Graph G mà có tất cả các nút được bao bởi số cạnh tối thiểu, một cây khung sẽ không hình thành một vòng tuần hoàn và nó cũng

4.4.2. Thuật toán duyệt đồ thị theo chiều sâu – DFS

Duyệt đồ thị theo chiều sâu DFS gần giống với kiểu duyệt cây nhị phân DLR, luôn đi từ nút xuất phát tới nút xa nhất cho tới khi không đi tiếp được nữa thì quay lại nút trước đó.

Cũng giống như BFS, ở đây ta sử dụng một danh sách Nut[] lưu các nút đã được duyệt và một ngăn xếp stack phục vụ cho quá trình duyệt. Trên mỗi bước lặp ta chọn một nút bất kỳ trong các nút liên kề với nút đang xét, lặp lại việc này cho đến lúc không đi tiếp được nữa; sau đó trở lại nút trước đó và tìm đường sang nút khác nếu còn. Khi một nút mà các nút kề nó không đi tiếp được nữa và đã đánh dấu duyệt rồi thì sẽ xóa nó khỏi ngăn xếp S. Thuật toán kết thúc khi stack đã rỗng.

Các bước của thuật toán DFS:

1. Đưa nút khởi đầu w vào ngăn xếp stack và vào Nut[];
2. while(!Empty(s)){
 - w = lấy ra nút ở nút ngăn xếp;
 - if(w đã duyệt) break;
 - Duyệt w; // thực hiện một thao tác nào đó với w
 - Đánh dấu w đã duyệt : đưa vào danh sách Nut[]
 - Tạo tập các nút liên kề v của w : V(w);
 - for (v ∈ V(w); v chưa được duyệt-chưa có trong Nut[]) đưa v vào stack;

Ta tiến hành các bước với đồ thị như sau :

Khởi tạo mảng dfs :

dfs:

--	--	--	--	--	--

Và ngăn xếp stack



Ta có thể bắt đầu từ một nút bất kỳ, ta chọn duyệt nút S – số hiệu nút là 0

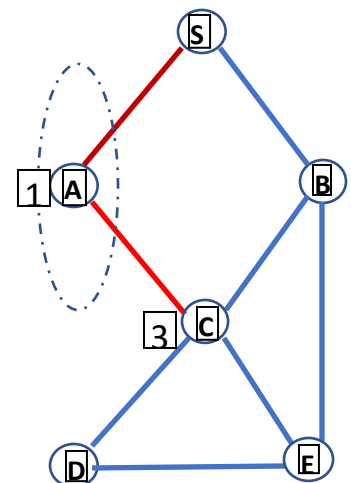
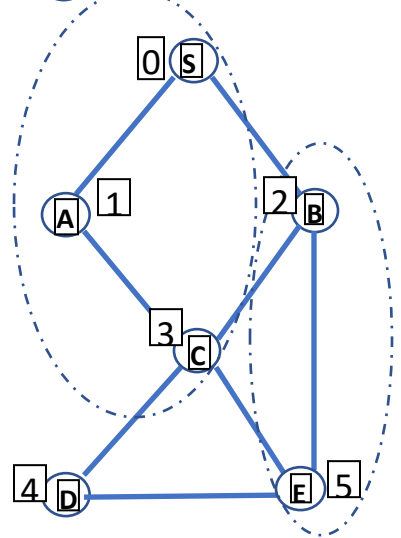
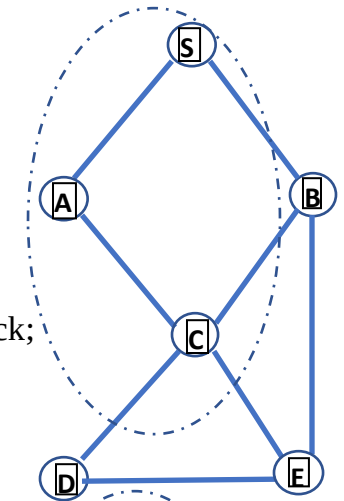
Cho i là số hiệu nút, cũng là chỉ số của mảng Nut[], Nut[i] chứa ký tự chữ tên nút; đặt k là số thứ tự duyệt, đồng thời để đếm số lần lặp DFS. Bắt đầu từ i = 0 ; gán Nut[0] = S ; k = 1; Đưa nút S vào stack

Lần lặp while thứ 1:

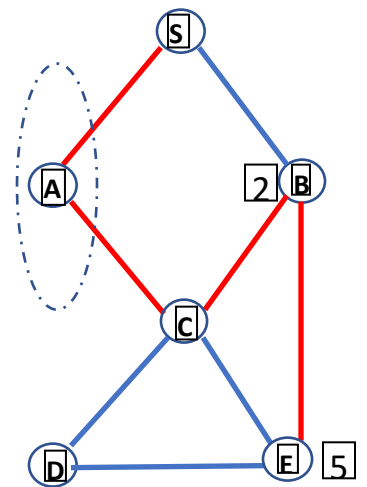
Lấy ra đỉnh ngăn xếp là S, xét tập nút liên kề của S = {A, B}, lấy 1 nút bất kỳ, lấy A, Nut[1] = A,	
Xét nút A chưa duyệt, đưa nút A vào stack	A
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	S

Lần lặp while thứ 2

Lấy ra đỉnh ngăn xếp là A, xét tập nút liên kề của A = {C}. Xét Nut[3] = C, nút C chưa duyệt, đưa C vào stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	C
	A
	S



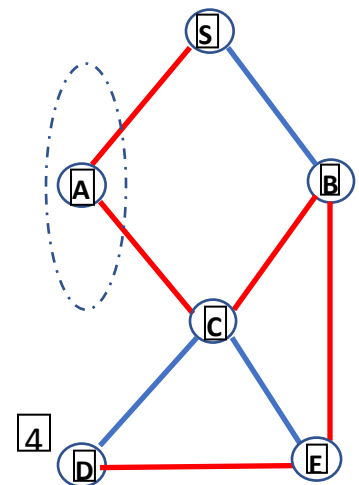
Lần lặp while thứ 3 :	
Lấy ra đỉnh ngăn xếp là C, xét tập nút liền kề của C = {A,B,E,D}. Xét Nut [2]= B	B
thấy nút B chưa duyệt, đưa B vào stack	C
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	A
	S



Lần lặp while thứ 4	
Lấy ra đỉnh ngăn xếp là B	E
Xét tập nút liền kề của B = {S,C,E}	B
Xét Nut [5]= E, nút E chưa duyệt, đưa E vào stack	C
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	A
	S

Lần lặp while thứ 5	
Lấy ra đỉnh ngăn xếp là E	D
Xét tập nút liền kề của E = {B,C,D}	E
Xét nút D, Nut [4]= D	B
nút D chưa duyệt, đưa D vào stack	C
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	A
	S

Lần lặp while thứ 6	
Lấy ra đỉnh ngăn xếp là D	E
Tất cả các nút liền kề với D đều đã duyệt nên xóa D ra khỏi stack	B
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	C
	A
	S



Lần lặp while thứ 7 :	
Lấy ra đỉnh ngăn xếp là E	B
Tất cả các nút liền kề với E đều đã duyệt nên xóa E ra khỏi stack	C
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	A
	S

Lần lặp while thứ 8 :	
Lấy ra đỉnh ngăn xếp là B	C
Tất cả các nút liền kề với B đều đã duyệt nên xóa B ra khỏi stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	A
	S

Lần lặp while thứ 9	
Lấy ra đỉnh ngăn xếp là C	A
Tất cả các nút liền kề với C đều đã duyệt nên xóa C ra khỏi stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	S

Lần lặp while thứ 10 :

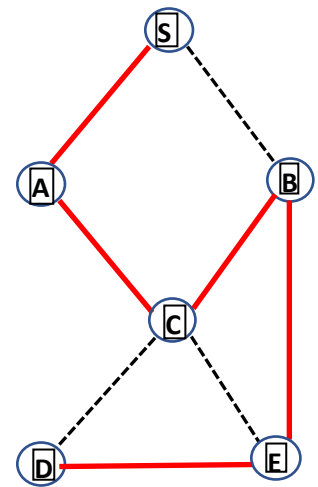
Lấy ra đỉnh ngăn xếp là A. Tất cả các nút liên kề với A đều đã duyệt nên xóa A ra khỏi stack	S
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp	

Lần lặp while thứ 11 :

Lấy ra đỉnh ngăn xếp là S. Tất cả các nút liên kề với S đều đã duyệt nên xóa S ra khỏi stack	
Kiểm tra, ngăn xếp rỗng, kết thúc lặp	

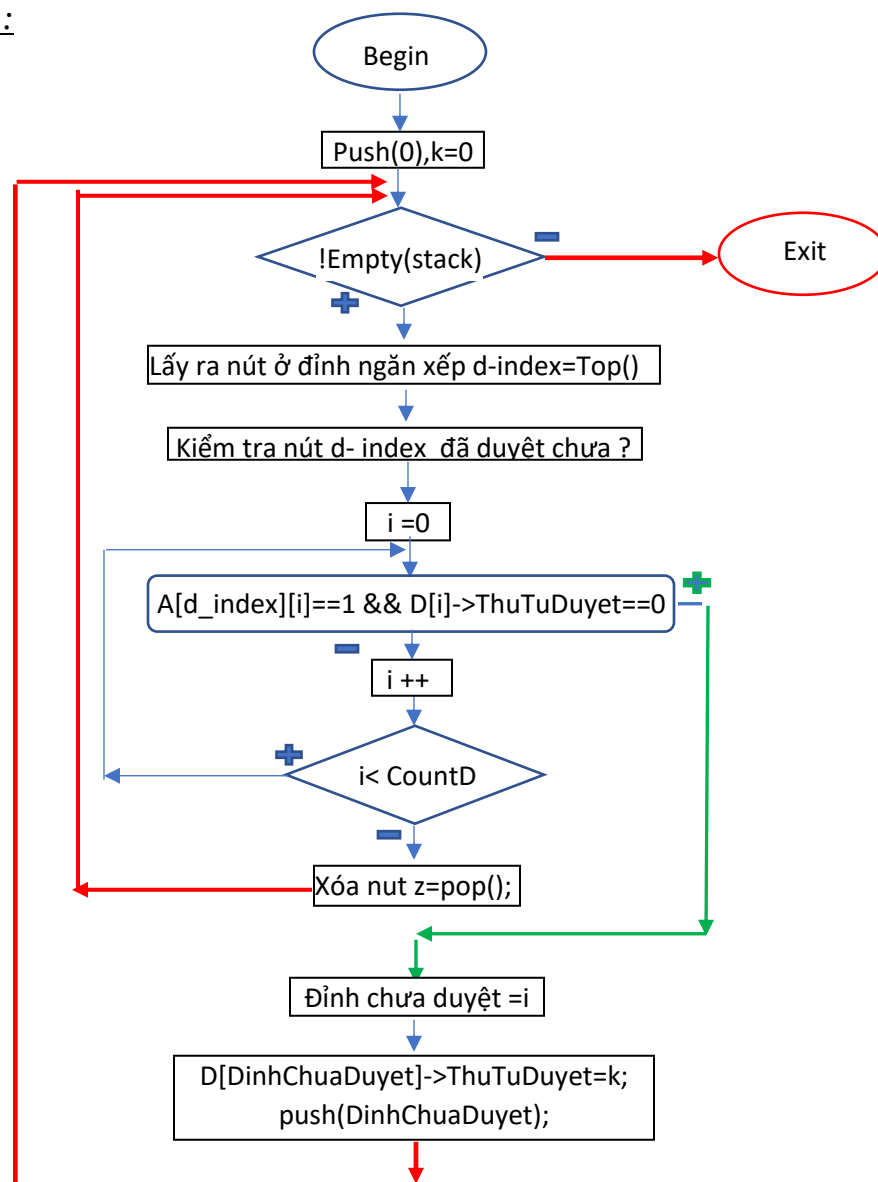
Mảng Nút cho kết quả là :

i =	0	1	2	3	4	5
Nut[i].dulieu =	S	A	B	C	D	E
Nut[i].ThuTuDuyet =	1	2	3	4	6	5



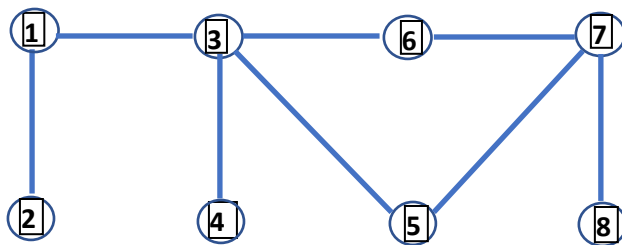
Kết quả duyệt cho một cây khung (spanning tree) S-A-C-B-E-D :

Sơ đồ thuật toán DFS :



Bài tập. 1) Đọc, tìm hiểu thuật toán duyệt đồ thị theo bề ngang và theo chiều sâu.

2) Cho đồ thị :



Hãy trình bày từng bước duyệt đồ thị theo bề ngang và theo chiều sâu

